S

SMARTHIRE AI

Project Report

An AI-powered resume analysis, job recommendation, and job-fit scoring system

Resume Parsing | Resume Classification | Job Recommendation | Fit Scoring | Job Clustering

Domain	AI / Machine Learning · Natural Language Processing
Core techniques	TF-IDF vectorization, Logistic Regression, Cosine Similarity, K-Means Clustering
Interface	Streamlit web application

1. Abstract

SmartHire AI is an end-to-end machine learning system that helps job seekers understand how their resume matches the job market. The system parses an uploaded resume, classifies it into a professional category, recommends the most relevant job openings from a large job corpus, and produces a fit score against any specific job description the user provides. A K-Means clustering module additionally groups job postings into broad domains for exploratory analysis. All components are built on TF-IDF text vectorization and are served through a Streamlit web application.

962	136,759	99.48%
Resumes analyzed (25 categories)	Job postings in the corpus	Classifier test accuracy

Figures above are taken directly from the project's training notebooks (see Section 6).

2. Introduction & Problem Statement

Job seekers typically face two disconnected problems: first, understanding which professional category or role their experience best fits, and second, finding which of the thousands of active job postings are actually relevant to their background. Manually searching and comparing job descriptions against one's own resume is time-consuming and error-prone, and most job boards offer only keyword search rather than genuine content-based matching.

SmartHire AI addresses this by combining a supervised text classifier with an unsupervised, content-based recommendation engine, so that a single resume upload can automatically produce a professional category, a ranked shortlist of matching jobs, and a quantified fit score against any job description of the user's choosing.

3. Objectives

- Automatically extract and structure text from uploaded resumes (PDF or DOCX).
- Classify a resume into its most likely professional category using supervised machine learning.
- Recommend the top-N most relevant job postings for a given resume using content-based similarity.
- Quantify how well a resume matches a specific job description, with an interpretable fit level.
- Explore the structure of the job market by clustering job postings into related domains.
- Present all of the above through an accessible, non-technical web interface.

4. Dataset Overview

Two primary datasets underpin the system: a labeled resume dataset used for classifier training, and a large job-postings corpus used for recommendation, fit scoring, and clustering.

4.1 Resume Dataset

Metric	Value
Total resumes	962
Unique professional categories	25
Train / test split	769 / 193 (80% / 20%, stratified)

4.2 Job Postings Corpus

Metric	Value
Total job postings	136,759
Unique companies	32,845
Unique locations	10,855
Unique experience-level labels	152

EDA showed that job opportunities are concentrated among a relatively small number of companies and locations, and that resume categories, while imbalanced, contain enough samples per class for reliable supervised classification.

5. System Architecture

The system follows a linear pipeline: a resume is parsed into raw text, that text is classified and matched against the job corpus, and the results are surfaced through the web interface. Each stage is implemented as an independent module so that models can be retrained or swapped without changing the others.

Stage	Module	Technique
1. Parsing	resume_parser.py	pdfplumber (PDF) / python-docx (DOCX)
2. Classification	classifier.py	TF-IDF + Logistic Regression
3. Recommendation	recommender.py	TF-IDF + Cosine Similarity (top-N ranking)
4. Fit Scoring	fit_predictor.py	TF-IDF + Cosine Similarity (resume vs. one job)
5. Clustering (exploratory)	clustering.py	K-Means on job TF-IDF vectors
6. Interface	streamlit_app.py	Streamlit multi-tab web application

6. Methodology

6.1 Resume Parsing

Uploaded files are routed by extension: PDF files are read page-by-page with pdfplumber and concatenated into a single text block, while DOCX files are read paragraph-by-paragraph with python-docx. The result is a single plain-text representation of the resume that feeds every downstream model.

6.2 Resume Classification

The classifier is trained on the 962-resume dataset. Resume text is vectorized with a TF-IDF vectorizer (English stop words removed, capped at 5,000 features), labels are encoded with a LabelEncoder across 25 categories, and a Logistic Regression model is trained on a stratified 80/20 split. The trained model, vectorizer, and label encoder are persisted (resume_classifier.pkl, tfidf_vectorizer.pkl, label_encoder.pkl) and loaded once at application start.

6.3 Job Recommendation Engine

All 136,759 job postings are vectorized with a separate TF-IDF vectorizer (5,000 features) fitted on the job text corpus. At request time, the resume is projected into the same vector space, cosine similarity is computed against every job vector, and the top-N most similar postings are returned along with their similarity score, company, and location.

6.4 Fit Predictor

The fit predictor reuses the job TF-IDF vectorizer to compare a single resume against a single user-supplied job description, again via cosine similarity. The resulting score is mapped to a qualitative fit level for interpretability:

Fit Score Range	Fit Level
≥ 80%	Excellent Match
60% – 79%	Good Match
40% – 59%	Moderate Match
< 40%	Low Match

6.5 Job Clustering (Exploratory)

To understand the broader structure of the job market, a K-Means model was trained on a random sample of 10,000 job postings (TF-IDF, 5,000 features). The Elbow Method indicated the within-cluster variance curve flattening around k = 5, so 5 clusters were selected for the final model. Inspecting the top TF-IDF terms per cluster centroid reveals distinct, human-interpretable domains:

Cluster	Top Terms	Inferred Domain
0	sales, store, customer, retail, service, manager	Sales & Retail

Cluster	Top Terms	Inferred Domain
1	doctorate, profile, software, specialization, programming	Software / Specialized Roles
2	project, data, engineering, design, business, technical	Data, Engineering & Project Management
3	care, patient, health, nursing, medical, clinical	Healthcare & Nursing
4	work, experience, team, accounting, skills, management	Accounting & General Business

This clustering step is exploratory and is not currently wired into the live recommendation flow; it provides a foundation for a future skill-gap or domain-tagging feature.

7. Results

7.1 Classifier Performance

On the held-out 193-resume test set, the Logistic Regression classifier achieved an overall accuracy of **99.48%**. Precision, recall, and F1-score were 1.00 for the large majority of the 25 categories, with the only imperfect class being *Automation Testing* (precision 0.83) and *DevOps Engineer* (recall 0.91) — both still resolved with high F1-scores (0.91 and 0.95 respectively). Macro-averaged F1-score across all categories was 0.99.

Such near-perfect accuracy is consistent with a small, cleanly labeled dataset with well-separated vocabulary between categories; performance on noisier, real-world resumes in production may be more modest and is worth monitoring over time.

7.2 Recommendation & Fit Scoring

Both the recommender and the fit predictor operate over the same 136,759-job, 5,000-feature TF-IDF space, so a resume's job recommendations and its fit score against any individual posting stay consistent with each other. Because the approach is entirely content-based (no user interaction history required), it works immediately for a first-time user with a single resume upload.

7.3 Clustering

The 5-cluster K-Means solution produced clearly separable, human-interpretable domains (Sales & Retail, Software, Data/Engineering/PM, Healthcare, and Accounting/General Business), confirming that the job corpus TF-IDF space captures meaningful semantic structure beyond just supporting similarity search.

8. Application Interface

The system is served through a single-page Streamlit application with a horizontal pill navigation bar and four sections:

- **Home** — product overview, a 3-step “how it works” explainer, and a summary of the three core capabilities.
- **Resume** — file upload (PDF/DOCX), extracted text preview, and predicted category. The most recently analyzed resume persists for the rest of the session so Jobs and Fit can reuse it without re-uploading.
- **Jobs** — top-5 recommended job postings for the analyzed resume, each shown with company, location, and similarity percentage.
- **Fit** — a free-text box to paste any job description, returning a fit score and qualitative fit level against the analyzed resume.

The interface uses a custom design system (navy / coral / teal palette, a circular “match ring” motif for scores) implemented via a standalone stylesheet layered on top of Streamlit's default components, keeping all presentation concerns separate from the underlying model code.

9. Technology Stack

Layer	Tools
Language	Python
Data handling	pandas, NumPy
NLP / Feature extraction	scikit-learn TfidfVectorizer
Modeling	scikit-learn (Logistic Regression, K-Means), cosine similarity
Resume parsing	pdfplumber, python-docx
Model persistence	joblib
Web interface	Streamlit, streamlit-option-menu
Experimentation	Jupyter notebooks (EDA, classifier, recommender, clustering, fit predictor)

10. Limitations & Future Work

- The classifier's near-perfect accuracy reflects a small (962-row), cleanly labeled dataset; a larger, more diverse resume sample would give a more realistic estimate of production performance.
- TF-IDF captures lexical overlap but not semantic meaning — two resumes describing similar skills with different wording may score lower than expected. Embedding-based similarity (e.g. sentence transformers) is a natural next step.
- Resumes that fail to extract text (e.g. scanned/image-only PDFs) currently flow through as empty strings rather than being explicitly flagged to the user.
- The K-Means clustering step is exploratory and not yet integrated into the recommendation or fit-scoring flow — a natural extension is a skill-gap feature that compares a resume's terms against its recommended cluster's top terms.
- Uploaded resume files are written to a temporary directory that is not currently cleaned up automatically.

11. Conclusion

SmartHire AI demonstrates a complete, practical pipeline for resume-driven job matching: parsing, supervised classification, content-based recommendation, and fit scoring, all built on well-understood, explainable TF-IDF and cosine-similarity foundations rather than opaque black-box models. The resulting classifier reached 99.48% test accuracy on the available resume dataset, the recommender and fit predictor operate consistently over a 136,759-job corpus, and the exploratory clustering step confirms the job corpus contains clear, interpretable domain structure. Together, these components are packaged behind a simple Streamlit interface that lets a user go from a raw resume file to a ranked shortlist of jobs and a quantified fit score in a few clicks.